MODULE 1: THE LIMITS OF LOGISTIC REGRESSION

Name: _____ Branch/Year: _____ Date: _____

> **THE SCENARIO:** "Experimental Drug Trial"
>
> We are analyzing the response data from a clinical trial of a new drug.
> *We want to build a model that can predict the result for any dosage.*

Task 1 — The Failure of the Curve

Step 1: Observe the patient data collected from the trial below.

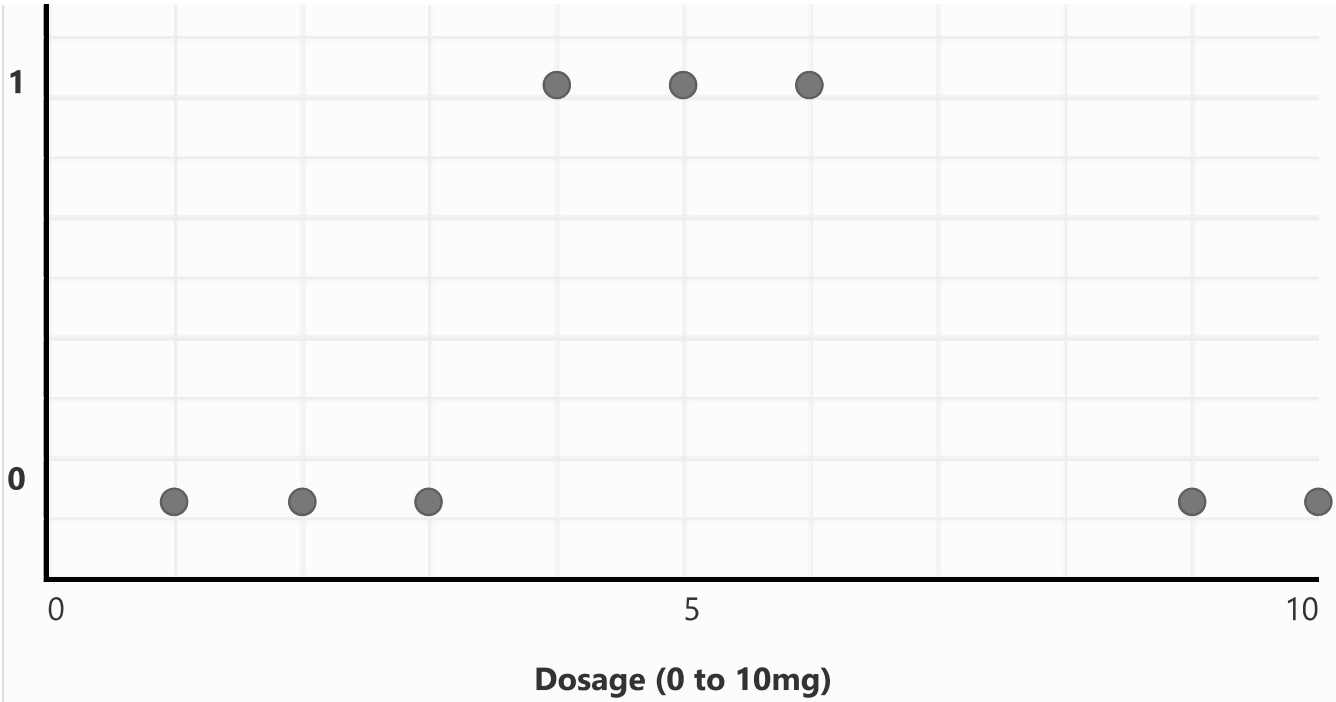
Patient ID	Dosage (X)	Result (Y) (0 = Sick, 1 = Cured)
1	1 mg	0
2	2 mg	0
3	3 mg	0
4	4 mg	1
5	5 mg	1
6	6 mg	1
7	9 mg	0
8	10 mg	0

Step 2: These points have been plotted on the graph for you.

Recall the "S-Curve" (Sigmoid) from Logistic Regression. It typically starts low and goes high (or starts high and goes low).

> **ACTIVITY: Try to draw a SINGLE Sigmoid (S-shaped) curve that passes through all these points correctly.**

Result (1=Cured, 0=Sick)



Step 3: Observation

Were you able to draw a single S-Curve that fits the data? Why or why not?

_____

_____

THE REVEAL: Why did the curve fail?
Here is the underlying logic:

- **Low Dosage:** Ineffective (0).
- **Medium Dosage:** Effective (1).
- **High Dosage:** Toxic/Ineffective (0).

> **CRITICAL FAILURE:** Logistic Regression assumes that "More X means More Probability of Y". It cannot handle data that goes UP and then comes back DOWN (non-monotonic data).

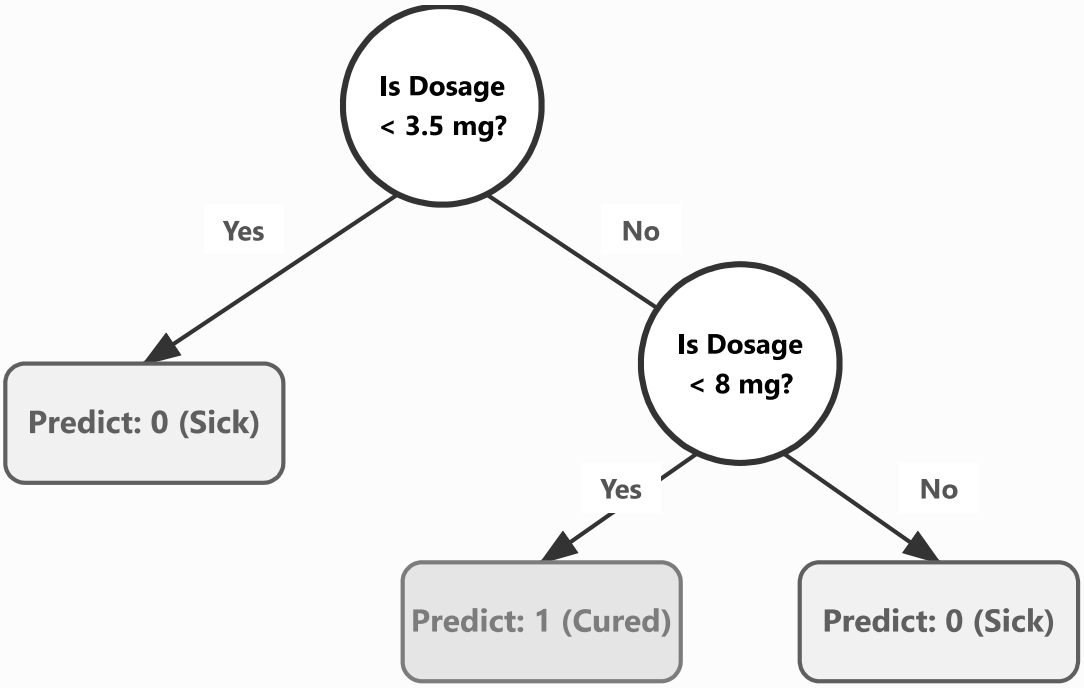
Task 2 — The Logical Solution

Since the math curve failed, let's use common sense logic.

Write your rule here (using Nested IF / ELSE logic):

```
IF Dosage is ________________________ THEN:
    → Predict: 0 (Sick)
ELSE:
    IF Dosage is ________________________ THEN:
        → Predict: 1 (Cured)
    ELSE:
        → Predict: 0 (Sick)
```

Visualize the Logic (Decision Tree):



MODULE 2: THE MATH OF DECISION TREES

THE SCENARIO: "Going Viral"

Predict if a reel will go **Viral (Yes/No)** based on:

- **Has Trending Audio?** (Yes / No)
- **Is Duration < 60s?** (Yes / No)

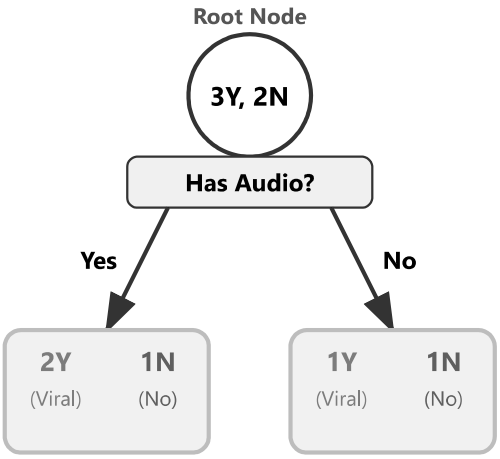
Task 1 — The Intuition Test

Step 1: Look at the dataset. Which feature will you choose to make the **split**?

Reel ID	Has Trending Audio?	Duration < 60s?	Went Viral? (Target)
1	Yes	Yes	**Yes**
2	Yes	Yes	**Yes**
3	No	Yes	**Yes**
4	Yes	No	**No**
5	No	No	**No**

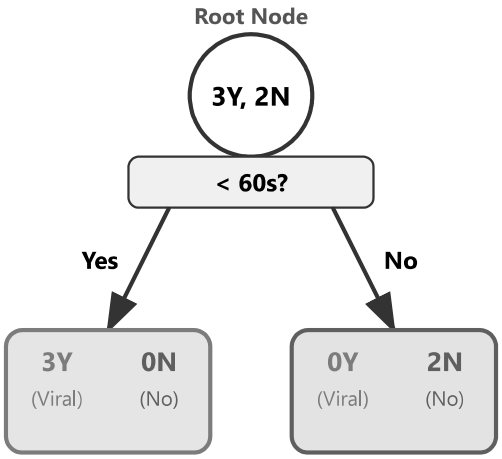
Step 2: Visualize the "Splits".

Option A: Splitting on Audio



(Messy!)

Option B: Splitting on Duration



(Pure!)

Which option does your "gut feeling" say is better?

My choice: _____

Task 2 & 3 — The Math (Gini Impurity)

$$\text{Gini} = 1 - (P_{Yes})^2 - (P_{No})^2$$

Option A: Splitting on Audio

1. Left Branch (Audio = Yes)
P(Yes) = _____ , P(No) = _____

Gini = 1 - (_____)^2 - (_____)^2 = _____ *(High Gini)*

2. Right Branch (Audio = No)
P(Yes) = _____ , P(No) = _____

Gini = 1 - (_____)^2 - (_____)^2 = _____ *(Max Gini)*

3. Final Score for 'Splitting on Audio' *(Weighted Average)*

Score = (0.6 × _____) + (0.4 × _____) = _____

Option B: Splitting on Duration

1. Left Branch (Duration < 60s)
Gini = 1 - (_____)^2 - (_____)^2 = _____ *(Pure)*

2. Right Branch (Duration > 60s)
Gini = 1 - (_____)^2 - (_____)^2 = _____ *(Pure)*

3. Final Score for 'Splitting on Duration' *(Weighted Average)*

Score = _____

WINNER: _____

Gini = 0.0	Perfect Purity (Goal)
Gini = 0.5	Maximum Mess (Enemy)

MODULE 3: BUILDING A COMPLETE DECISION TREE

> **THE SCENARIO:** "The Zomato Decision"
> Predict **Order Food (Yes/No)** based on **Hungry?** and **Coupon?**.

The Data (10 Days)

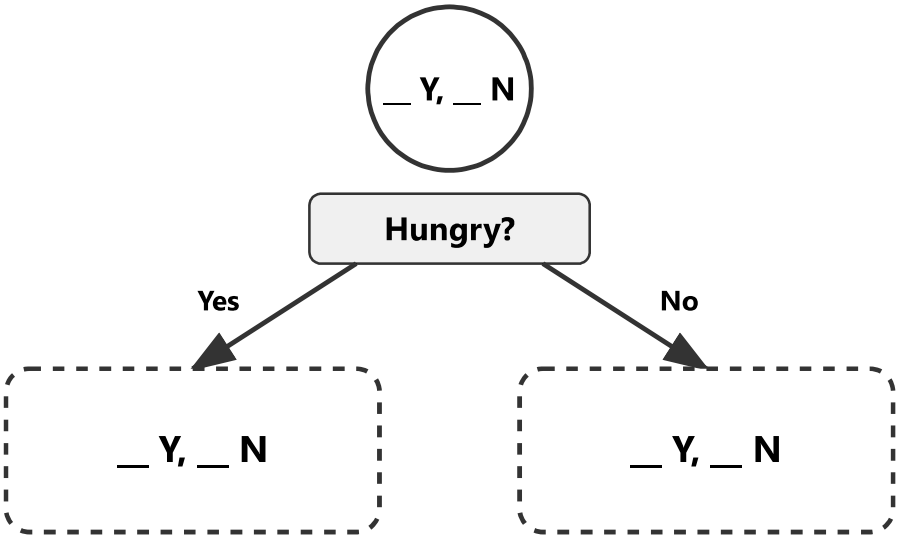
Day	Hungry?	Coupon?	Ordered?
1	No	Yes	No
2	No	Yes	No
3	No	Yes	No
4	No	No	No
5	No	No	No
6	No	No	No
7	Yes	No	No
8	Yes	No	No
9	Yes	Yes	**Yes**
10	Yes	Yes	**Yes**

Total: 2 Yes, 8 No

Step 1: Finding the Root Node

INSTRUCTIONS: Count the "Yes" and "No" for each branch and fill in the blanks.

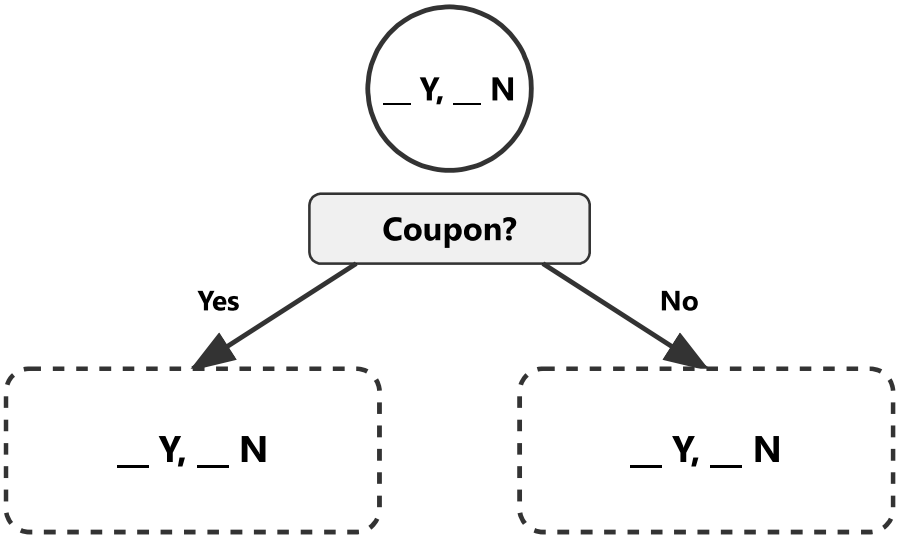
Option A: Splitting on "Hungry?"



Gini Calculation (Hungry):

1. No Branch: Gini = 1 - (_____)^2 - (_____)^2 = _____
2. Yes Branch: Gini = 1 - (_____)^2 - (_____)^2 = _____
3. Weighted: (____ × ____) + (____ × ____) = _____

Option B: Splitting on "Coupon?"

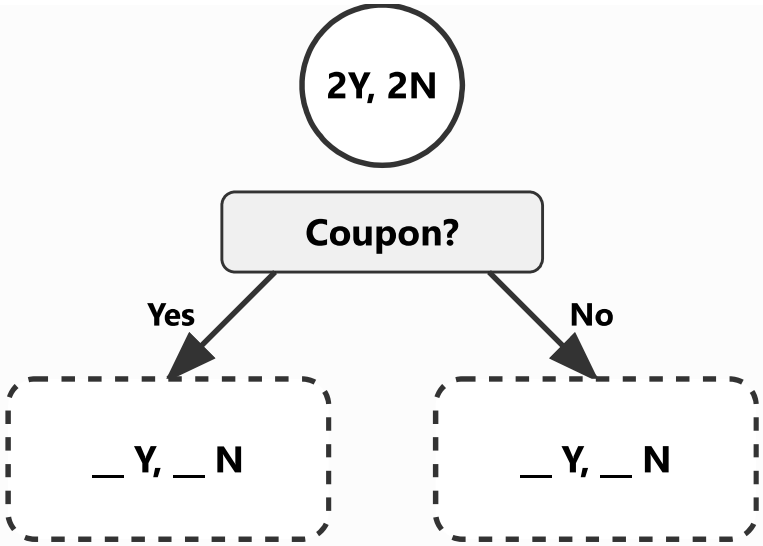


Gini Calculation (Coupon):

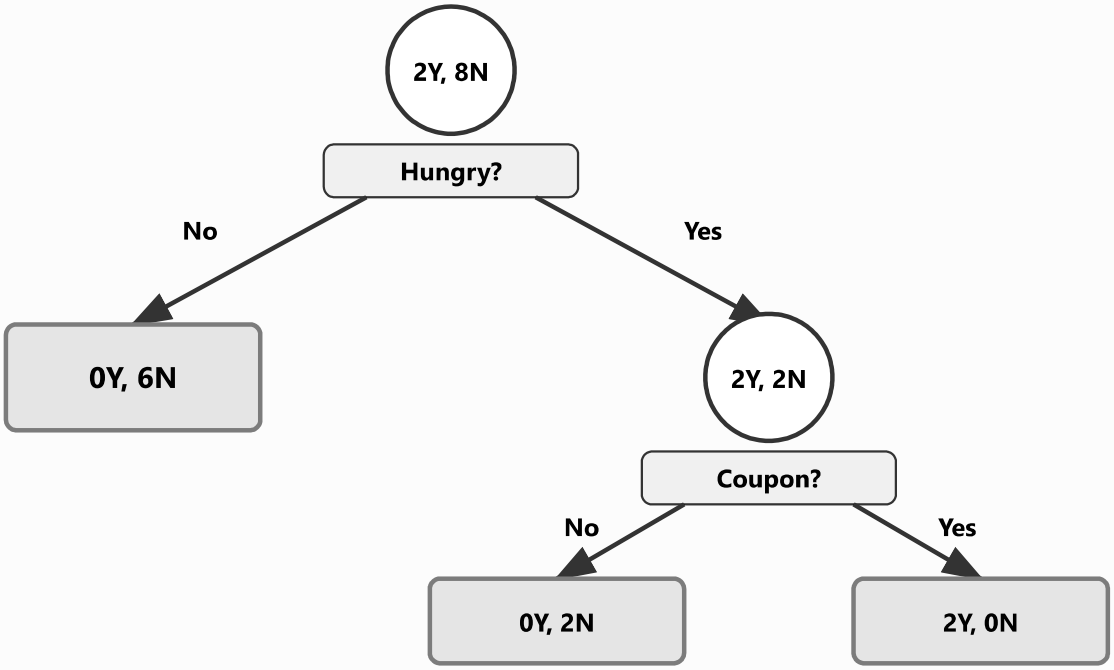
1. No Branch: Gini = 1 - (_____)^2 - (_____)^2 = _____
2. Yes Branch: Gini = 1 - (_____)^2 - (_____)^2 = _____
3. Weighted: (____ × ____) + (____ × ____) = _____

Step 2: The Second Split

The **"Hungry = Yes"** branch is still mixed (2 Yes, 2 No). Split it using **Coupon**.



Step 3: The Complete Tree



MODULE 4: IMPLEMENTING DECISION TREES IN PYTHON

OBJECTIVE: In this module, we will write a complete Python script to predict if a customer will buy an SUV. We will build a basic model, observe overfitting, and then improve it using Hyperparameter Tuning.

Step 1: Import Necessary Libraries

[ ] **Code Cell 1**

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.tree import DecisionTreeClassifier, plot_tree
from sklearn.model_selection import train_test_split, GridSearchCV
from sklearn.metrics import accuracy_score
```

Step 2: Load the Dataset

[ ] **Code Cell 2**

```
# 1. Load the dataset
df = pd.read_csv('data.csv')

# 2. View the data
df
```

Step 3: Separate Features and Target

[ ] **Code Cell 3**

```
# 1. Separate Features (X)
# We select 'Age' and 'Salary' as our input columns
X = df[['Age', 'Salary']]

# 2. Separate Target (y)
# We select 'Purchased' as the target to predict
y = df['Purchased']
```

Step 4: Split into Training and Testing

[ ] **Code Cell 4**

```
# Split data: 80% Train, 20% Test
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2,
random_state=42)
```

Step 5: Train the Initial Model

[ ] **Code Cell 5**

```
# Initialize the model without any limits (Unconstrained)
model = DecisionTreeClassifier()

# Train the model on the Training data
model.fit(X_train, y_train)
```

Step 6: Visualize the Tree

[ ] **Code Cell 6**

```
# Set the figure size to make it readable
plt.figure(figsize=(12, 8))

# Plot the tree
plot_tree(model,
          feature_names=['Age', 'Salary'],
          class_names=['Not Purchased', 'Purchased'],
          filled=True)

plt.show()
```

Step 7: Check for Overfitting

[ ] **Checking accuracy on training set**

```
accuracy_score(y_train, model.predict(X_train))
```

[ ] **Checking accuracy on testing set**

```
accuracy_score(y_test, model.predict(X_test))
```

OBSERVATION: Notice that Training Accuracy is very high (often 100%), but Testing Accuracy is lower.
This is called **OVERFITTING**. The model memorized the training data instead of learning the pattern.

Step 8: Hyperparameter Tuning (Manual)

To fix overfitting, we use **Hyperparameters** (e.g., `max_depth`) to limit the tree's growth.

[ ] **Code Cell 8**

```
# Change the model = DecisionTreeClassifer()
model = DecisionTreeClassifier(max_depth=2)

# Train again
model.fit(X_train, y_train)

# Check accuracy_score on training and testing using above code
```

Step 9: More Hyperparameters

We can control the tree even more precisely using `min_samples_split` and `min_samples_leaf`.

[ ] **Code Cell 9**

```
# Using multiple hyperparameters to fine-tune
model = DecisionTreeClassifier(max_depth=2, min_samples_split=10,
min_samples_leaf=5)

model.fit(X_train, y_train)
```

Step 10: Automated Tuning (Grid Search)

Instead of guessing values manually, we use **GridSearchCV** to try every combination and find the best one.

[ ] **Code Cell 10**

```
# 1. Define the grid of parameters to test
param_grid = {
    'max_depth': [2, 3, 4, 5],
    'min_samples_split': [2, 5, 10],
    'min_samples_leaf': [1, 2, 5]
}

# 2. Setup Grid Search (cv=5 means 5-fold Cross Validation)
grid = GridSearchCV(DecisionTreeClassifier(), param_grid, cv=5)

# 3. Fit (This runs many models to find the winner)
grid.fit(X_train, y_train)

# 4. View the best parameters found
print("Best Params:", grid.best_params_)
```